

Software Implementation and Testing Document

For

Group <Closet Design Game>

Version 1.0

Authors:

Ethan McConnell
Wesley Jordan
Vanessa Campoe
Camryn Morris
Chloe Patrick
Jessika Tyler

1. Programming Languages (5 points)

JavaScript/TypeScript was used for the react application logic, UI events, Fabric.js canvas integration, brush controls, and garment placement because it was the easiest primary language family that suited our web application and had the libraries included in our proposal. HTML was used for browser document structure and rendering context used by the React application because it is the standard structure for a browser-based single page application. CSS/Tailwind CSS was used for studio layout, navigation bar, side drawers, controls, responsive presentation because it provides efficient utility-based styling for the studio UI. Python was used for development-time script that resizes SVG garment assets and normalizes their center/anchor points because it is well suited to repeatable batch file processing and asset preparation. SVG is used for vector garment/mannequin assets and clipping boundaries consumed by the canvas because vector paths scale cleanly and can provide precise garment shapes for alignment and clipping.

2. Platforms, APIs, Databases, and other technologies used (5 points)

React: Used for the studio page, navigation, slide-out drawers, clothing-category controls, and reactive UI state.

Vite: Used as the frontend development/build tool for the React application.

Fabric.js: Used for the interactive design canvas, freehand drawing, brush settings, garment objects, and clipping masks.

Tailwind CSS: Used to implement the main page/studio layout and style the UI controls.

Lucide React: Used for placeholder and studio-control icons in the navigation/drawers.

Static JSON catalog: Used to categorize garment data by era/decade, geographic region, clothing type, and asset path.

No database is required for Increment 1.

GitHub: Used for source control and collaboration; repository rules require review before changes are merged.

Browser HTML5 Canvas/SVG support: Provides the runtime graphics environment used by Fabric.js and the vector garment assets

3. Execution-based Functional Testing (10 points)

Functional testing was performed by running the program and seeing if the desired functional requirements were met. Doing so, it can be found that when ran, the program correctly displays a blank, centered design canvas after the studio page loads. The canvas has a fixed width and height, so the working area remains consistent. The program allows users to perform freehand strokes across the screen. The width of the brush can be adjusted with a slider. The program allows a user to select a brush stroke. The opacity of the brush can be adjusted. The program provides an eraser mode for removing user-created strokes. The program provides slide-outs for browsing and selecting eras and geographic regions. The program provides selectable clothing categories for tops, bottoms, and hats. The system currently doesn't place the garment item once an available one is selected. The garment isn't placed, so it can't be aligned to the common canvas/mannequin anchor. Visible paint strokes can bleed into the background. The clothing catalog data shall identify each garment by its era, geographic region, clothing category, and asset reference so the interface filter and retrieve items. The studio page provides a top navigation bar and side-drawer controls needed to access the design functions.

4. Execution-based Non-Functional Testing (10 points)

Non-functional testing was also performed by running the code and testing whether the non-functional requirements were met. By running the program, it was found that the increment 1 system runs as a web-based single-page application that requires no native installation. During testing, the application and basic canvas component didn't crash. The size of the canvas remains the same even if the window size changes. The brush is responsive enough to maintain normal

interactivity. The preprocessed SVG garment assets weren't placed onto the canvas; therefore, it can't be determined if they appeared in predictable positions through execution-based testing. Changes merged into the shared repository were reviewed by at least one other team member before being merged.

5. Non-Execution-based Testing (10 points)

Non-execution-based testing was done by having a team member review another member's code before merging it into the main branch. Doing this, it has been determined that the preprocessed SVG garment assets should appear in predictable positions when placed.